Tutorial 3 | Séries temporais com LSTM

Previsão mensal de nascimentos sintéticos · Explicação técnica para o docente

1. Pergunta de pesquisa e unidade temporal

O terceiro tutorial apresenta uma rede neural recorrente do tipo LSTM (Long Short-Term Memory) na previsão de nascimentos mensais. A pergunta didática é: como utilizar observações de meses anteriores para estimar o número de nascimentos do mês seguinte? A aplicação foi escolhida para conectar Deep Learning a uma estrutura familiar em estudos populacionais: uma série temporal com tendência e sazonalidade.

A base contém 120 observações mensais sintéticas de janeiro de 2015 a dezembro de 2024. Cada valor é produzido somando um nível inicial de 2.500, uma tendência linear de redução de 2,5 por mês, uma oscilação senoidal de amplitude 170 e um termo aleatório normal com desvio-padrão 35. A combinação simula características visuais de uma série, mas não descreve processos reais de fecundidade, cobertura de registros ou sazonalidade de nascimentos.

```
nascimentos(t) = 2500 - 2.5*t
               + 170*sin(2*pi*t/12) + ruído(t)
```

2. Bibliotecas: funções efetivamente usadas

- NumPy: constrói o índice numérico dos meses, calcula a função seno e gera o ruído; seus arrays armazenam janelas de sequências e entradas da LSTM.
- pandas: date_range(..., freq="MS") produz datas de início de mês; Series associa as contagens aos meses; a indexação temporal permite recuperar datas correspondentes ao teste.
- Matplotlib: gera o gráfico da série original, as curvas de perda durante treinamento e o gráfico com valores observados, previsões LSTM e referência sazonal.
- scikit-learn / MinMaxScaler: aplica transformação linear usando mínimos e máximos observados somente no treino. transform normaliza a série; inverse_transform devolve previsões para a escala de nascimentos.
- scikit-learn / mean_absolute_error: calcula o erro absoluto médio (MAE) em unidades da série. Ao contrário da perda de treinamento, esta avaliação final ocorre na escala original dos nascimentos.
- TensorFlow/Keras: Sequential organiza as camadas; Input define o formato; LSTM(16) processa a sequência com 16 unidades; Dense(1) produz uma previsão; compile configura otimizador e perda; fit treina; predict gera estimativas.

3. Séries temporais, tendência, sazonalidade e ruído

Tendência é uma mudança gradual de nível ao longo do tempo. Sazonalidade é uma variação que se repete aproximadamente após intervalos fixos (no exemplo, 12 meses). Ruído representa flutuações não descritas pelas partes determinísticas. O modelo não recebe essas etiquetas; observa apenas valores numéricos, de modo que o professor pode discutir o que uma rede precisa aprender para prever a série.

4. Por que a divisão treino-teste é cronológica

Em séries temporais, um sorteio aleatório entre meses pode permitir que o modelo seja ajustado com observações posteriores ao período que se pretende prever. Para evitar isso, o notebook reserva os primeiros 96 meses para treino e os últimos 24 meses para teste. Não é uma validação externa, mas respeita a ordem dos acontecimentos.

O escalonador é estimado com valores[:96]. Ajustar a transformação usando os 120 meses faria com que informações do intervalo futuro influenciassem o pré-processamento: uma forma de vazamento temporal. Depois de ajustar no treino, a mesma transformação é aplicada ao período completo para formar janelas, incluindo dados anteriores necessários a cada previsão.

5. Janelas móveis: transformar uma série em exemplos

Uma LSTM recebe sequências, não apenas uma observação isolada. O notebook define janela = 12: para prever um mês t, utiliza os valores dos meses t-12 a t-1. Assim, 120 meses produzem 108 janelas, das quais 84 são de treinamento e 24 de teste. O array X possui três dimensões: número de exemplos, passos temporais e variáveis por passo.

```
X.shape: (108, 12, 1)
X_treino.shape: (84, 12, 1)
X_teste.shape: (24, 12, 1)
```

O código identifica o mês-alvo de cada janela com um índice i. As janelas de treino são aquelas para as quais o mês a prever ocorre antes do mês 96. No teste, os 12 valores anteriores podem incluir meses do próprio período de teste, porque se trata de previsão de um passo à frente com atualização das observações históricas, e não previsão simultânea dos 24 próximos meses.

6. O que é uma LSTM e como funciona

Redes recorrentes processam informação ordenada, atualizando um estado ao longo da sequência. A LSTM acrescenta mecanismos chamados portas (gates) e um estado de célula para controlar o que preservar, atualizar ou expor. As portas de esquecimento, entrada e saída usam funções aprendidas a partir dos dados. Isso ajuda a modelar dependências temporais, embora não elimine problemas de pouca informação, sobreajuste e instabilidade.

No modelo proposto, Input(shape=(12, 1)) indica 12 meses e uma característica por mês. LSTM(16) cria uma representação da sequência com 16 unidades; Dense(1) transforma essa representação em uma estimativa escalar. A arquitetura é deliberadamente pequena para fins didáticos e pode ser executada em CPU.

7. Treinamento: perda, épocas e validação

O otimizador Adam ajusta pesos iterativamente, usando estimativas adaptativas dos gradientes. A função de perda MSE (erro quadrático médio) penaliza mais fortemente erros elevados e é computada sobre dados normalizados. Cada época representa uma passagem de treinamento pelos exemplos; o notebook usa 25 épocas e lotes de 16 sequências.

O argumento validation_split=0.15 reserva uma parte final do conjunto de janelas de treino para calcular a perda de validação. shuffle=False evita embaralhar a ordem dos exemplos, mas janelas temporais próximas compartilham vários meses: portanto, a validação interna não deve ser confundida com uma avaliação rigorosa de previsão em origens temporais múltiplas. Em aplicações reais, recomenda-se validação temporal progressiva.

8. Avaliação: LSTM versus referência sazonal

Após treinar, o notebook calcula previsões para os 24 meses de teste e aplica inverse_transform. O MAE é a média dos valores absolutos das diferenças entre contagens previstas e observadas. Se o MAE for 100, isso significa um erro absoluto médio de 100 nascimentos na série sintética.

```
MAE = soma_t |observado(t) - previsto(t)| / N
```

A linha de base sazonal ingênua prevê para o mês t o mesmo número observado no mês t-12. É um comparador importante porque incorpora sazonalidade anual sem treinar uma rede neural. O notebook imprime os dois MAEs e exibe um gráfico comparativo. Se a LSTM perder para a referência, isso não é erro no tutorial: significa que um modelo mais complexo não demonstrou vantagem naquele teste.

9. Fala orientadora para cada etapa

- Passo 1: apresente Keras como interface de construção de redes e TensorFlow como ambiente de cálculo. Mostre que NumPy, pandas e Matplotlib continuam úteis em Deep Learning.
- Passo 2: identifique a direção da tendência e os ciclos anuais no gráfico, lembrando que foram introduzidos artificialmente.
- Passo 3: mostre uma janela de 12 meses e a observação imediatamente seguinte. Pergunte por que não convém treinar com o futuro.
- Passo 4: explique Input, LSTM e Dense. Compare as curvas de perda e esclareça que reduzir a perda no treino não garante melhor previsão.
- Passo 5: interprete MAE na unidade original, compare com a previsão sazonal e discuta a parcimônia: a complexidade deve trazer ganho verificável.

10. Limitações e recomendações para estudos populacionais

O exercício usa apenas 84 janelas de treino, muitas delas altamente sobrepostas. Isso é pouco para sustentar inferências amplas sobre desempenho de redes neurais. Em bases reais, é necessário investigar rupturas estruturais, mudanças de cobertura, efeitos de calendário, revisão de dados, escolha do horizonte de previsão e sazonalidade. Outras estratégias, como modelos sazonais tradicionais, regressões com componentes temporais e previsões de referência, devem participar da comparação.

Perguntas para discussão

- Por que prever um mês com os 12 meses anteriores difere de prever 24 meses futuros de uma só vez?
- O que revela uma perda de treino decrescente e uma perda de validação crescente?
- Quando uma referência sazonal simples seria preferível a uma LSTM?

Referências para aprofundamento

Hochreiter, S.; Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

Hyndman, R. J.; Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd ed. OTexts. https://otexts.com/fpp3/

Documentação oficial: https://keras.io/api/layers/recurrent_layers/lstm/ e https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html

Correspondência com a prática: tutorial de 30 minutos publicado em cbeluzo.github.io/abep-2026/tutoriais/pratica-2h/. Material explicativo; os dados dos exemplos são sintéticos.